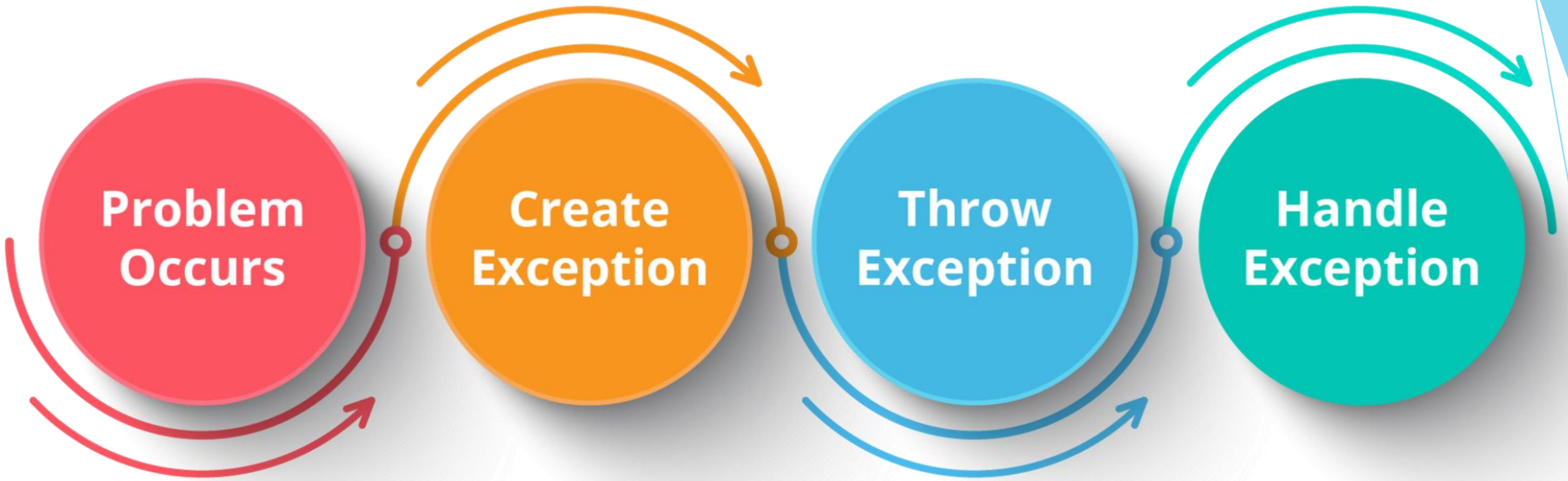
BCA Sem - 3
Programing using Java

Unit - 3
Exception Handling





Exception Handling in Java

The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained

What is exception ?

Dictionary Meaning: Exception is an abnormal condition.

In java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Introduction

During the execution of an application, there are many times when an error condition arises within the application. In java, all kinds of error conditions are called as exceptions. In other words we can say that **In General, it is in the rare case that our program will run successfully in its first attempt.**

There are mainly two types of exceptions :
(1)Compile Time Exceptions (Syntax Errors)
(2)Run Time Exceptions (Logical Errors)

Introduction

(1) Compile Time Errors :

All the syntax Errors are known as Compile Time Errors. These types of errors are detected and displayed by the java compiler therefore they are known as Compile Time Errors. This type of errors can be solved before running the program so that after solving this type of errors we are sure that our program is syntactically correct.

Remember that : Whenever the compiler will display the error it will not generate the .class file.

Introduction

(1) Compile Time Errors Examples :

- Missing Semicolons
- Missing Brackets
- Misspell of Keywords of Variables
- Missing of double quotes in strings
- Use of undeclared variables
- Incompatible type conversion / assignment
- use of = inplace of ==
- use of non initialized variable in keyboard reading

Introduction

(2) Run Time Errors :

Run time errors are those types of errors in which the program may compiled successfully and the .class file also generated but when we run that program it will produce the wrong result.

This will happened due to poor understanding of the logic or sometimes due to misuse of the system resources.

Introduction

(2) Run Time Errors Examples :

- Dividing an integer by Zero
- Accessing the element out of bounds of Array
- Trying to store the value of incompatible type into array
- Passing the parameter which is not compatible for the method.
- Trying to illegally change the state of Thread.
- Use of Negative size of Array.
- Conversion of invalid string to number

Advantage of Exception Handling

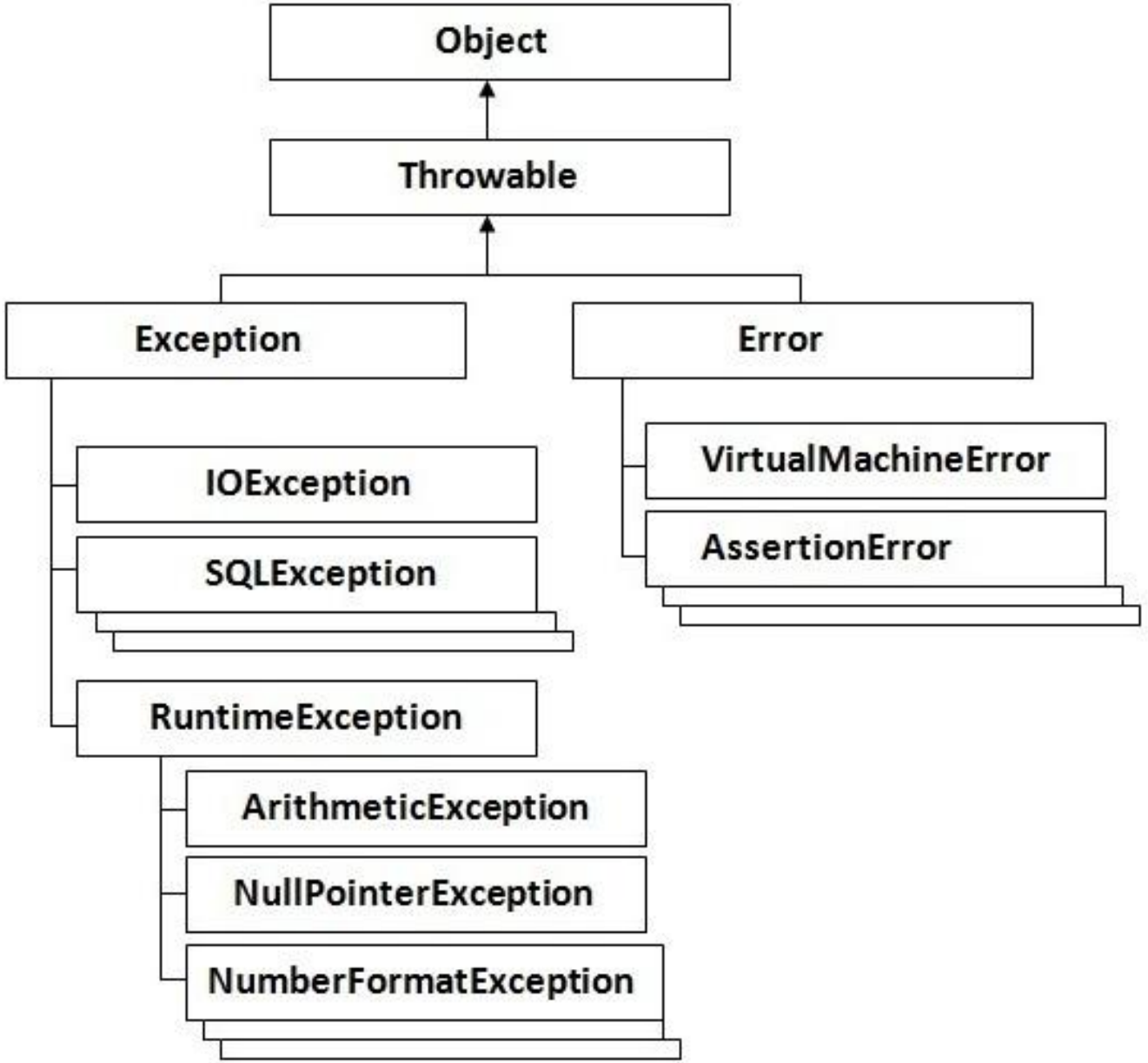
The core advantage of exception handling is **to maintain the normal flow of the application.** Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

```
statement 1;
statement 2;
statement 3;
statement 4;
statement 5;//exception occurs
statement 6;
statement 7;
statement 8;
statement 9;
statement 10;
```

In previous example, Suppose there is 10 statements in your program and there occurs an exception at statement 5, rest of the code will not be executed i.e. statement 6 to 10 will not run.

If we perform exception handling, rest of the statement will be executed. That is why we use exception handling in java.

Hierarchy of Java Exception classes



Types of Exception

There are mainly two types of exceptions: checked and unchecked where error is considered as unchecked exception.

The sun micro system says there are three types of exceptions:

Checked Exception

Unchecked Exception

Error

1) Checked Exception

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions

e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

1) Checked Exception

```
import java.io.*;
class JTUAddition
{
    public static void main(String args[])
    {
      DataInputStream dis=new DataInputStream(System.in);
      int a=0,b=0,sum=0;
      System.out.print("Enter Value 1 :");
      a=Integer.parseInt(dis.readLine());
      System.out.print("Enter Value 2 : ");
      b=Integer.parseInt(dis.readLine());
      sum=a+b;
      System.out.println("Addition is : "+sum);
    }
}
```

1) Checked Exception

```
import java.io.*;
class JTUAddition
{
    public static void main(String args[])
    {
      DataInputStream dis=new DataInputStream(System.in);
      int a=0,b=0,sum=0;
      try
      {     System.out.print("Enter Value 1 :");
            a=Integer.parseInt(dis.readLine());
            System.out.print("Enter Value 2 : ");
            b=Integer.parseInt(dis.readLine());
            sum=a+b;
            System.out.println("Addition is : "+sum);
    }
```

1) Checked Exception

```
}
        catch(Exception e)
        {
                System.out.println("Error Occurred");
        }
}
```

2) Unchecked Exception

The classes that extend RuntimeException are known as unchecked exceptions

e.g. ArithmeticException,

NullPointerException,
ArrayIndexOutOfBoundsException etc.

Unchecked exceptions are not checked at compile-time rather they are checked at runtime.

2) Unchecked Exception

```
class Main {
  public static void main(String args[]) {
    int x = 0;
    int y = 10;
    int z = y/x;
  }
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero at
    Main.main(Main.java:5) Java Result: 1
```

Exception Handling

Generally an Exception is an error caused by run-time error in the program.

If this types of errors (run time errors) are not handled properly the interpreter will generate the error as shown in previous example.

To handle this situation java has provided us a special mechanism called Exception Handling.

The purpose of this mechanism is to handle the exceptional circumstances.

Exception Handling

The mechanism for Exception handling can be summarize in the following steps:

(1) Find the problem (Hit)

(2) Inform that an error has occurred (Throw)

(3) Receive the error information (Catch)

(4) Take the corrective Actions (Handle)

For this purpose java has given us various built in Exceptions Classes to handle some common exceptions.

Some Common Java Exceptions

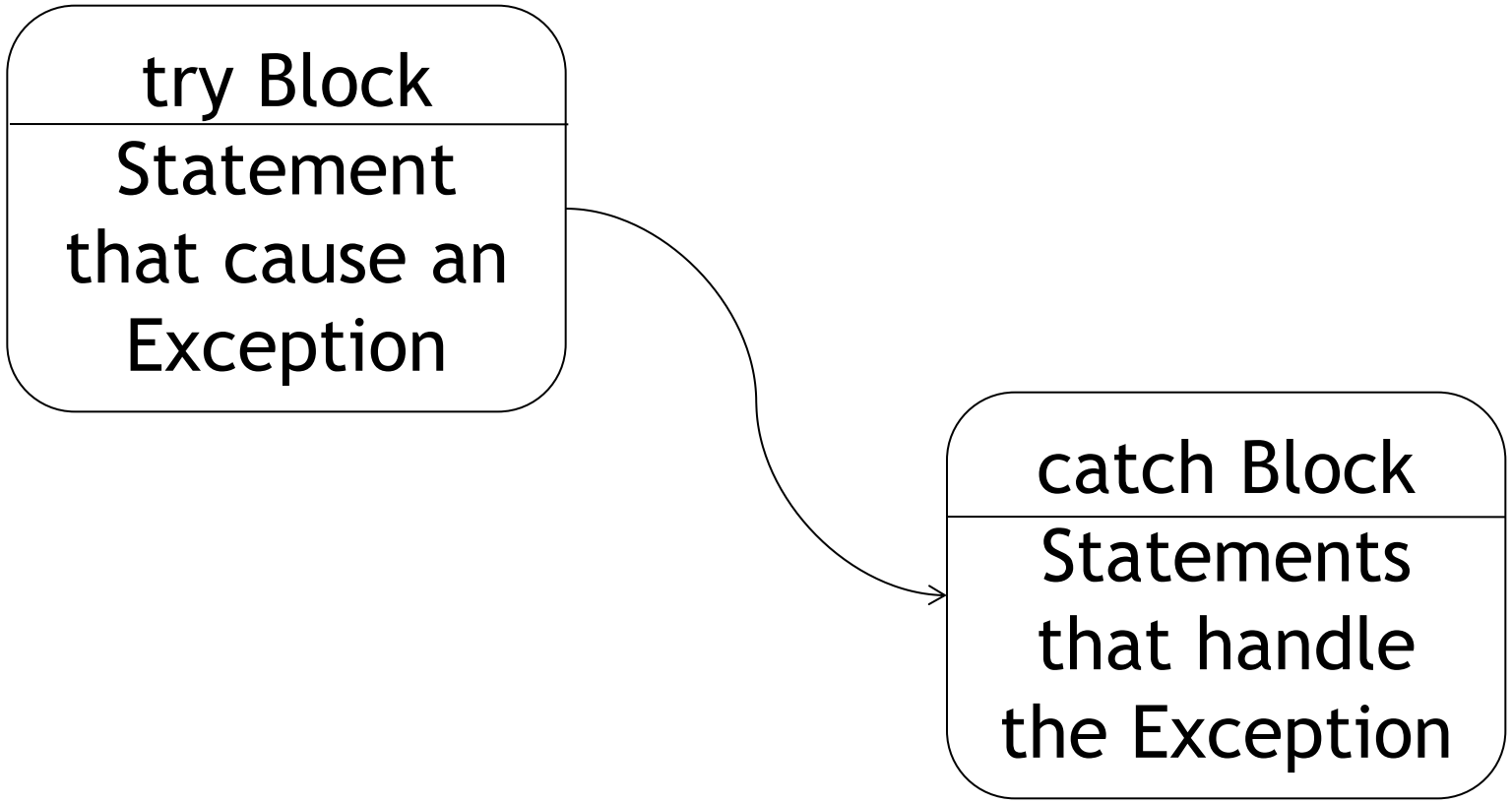
Exception	Cause of Exception
ArithmeticException	Caused by mathematical Errors such as division by Zero
ArrayIndexOutOfBoundsException	Caused by bad array indexes
ArrayStoreException	Caused when program try to store wrong type of data in array.
FileNotFoundException	Caused by an attempt to access a nonexistent file.
IOException	Caused by general I / O failures.
NullPointerException	Caused by referencing to a null object
NumberFormatException	Caused when conversion between string and number fails

Some Common Java Exceptions

Exception	Cause of Exception
OutOfMemoryException	Caused when there is no enough memory to allocate a new object
SecurityException	Caused when an applet tries to perform an action not allowed by the browser's security settings.
StackOverFlowException	Caused when the system runs out of stack space
StringIndexOutOfBoundsException	Caused when a program attempts to access a non existent character position in a string.

Exception Handling Mechanism

Following figure can illustrate the Exception Handling Mechanism



Java Exception Handling Keywords

There are 5 keywords used in java exception handling.

try

catch

finally

throw

throws

Java try-catch

* Java try block

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

Java try block must be followed by either catch or finally block.

Syntax of java try-catch

```
try{
//code that may throw exception
} catch(Exception_class_Name ref){}
```

Java catch block

Java catch block is used to handle the Exception.

It must be used after the try block only.

You can use multiple catch block with a single try.

Problem without exception handling

Let's try to understand the problem if we don't use try-catch block.

```
public class Testtrycatch1{
  public static void main(String args[]){
     int data=50/0;//may throw exception
     System.out.println("rest of the code...");
    }
}
```

Output:

Exception in thread main

java.lang.ArithmeticException:/ by zero

Let's see the solution of above problem by java try-catch block.

```
public class Testtrycatch2{
  public static void main(String args[]){
    try{
      int data=50/0;
    }
    catch(ArithmeticException e){System.out.println(e);}
    System.out.println("rest of the code...");
    }
}
```

Output:

```
Exception in thread main
java.lang.ArithmeticException:/ by zero
rest of the code...
```

Use of Multiple Catch Statements

In java we can use multiple catch statements if we think that particular statement block will generate multiple type of errors.

e.g. if we think that a group of statements i.e. a block will generate either divide by zero or array index out of bounds errors then we can use multiple catch statements.

Use of Multiple Catch Statements

```
class ex46
{
	public static void main(String args[])
	{
		int a[]={5,10};
		int b=5;

		try
		{
			int x=a[2]/(b-a[0]);
		}
		catch(ArithmeticException e)
		{
			System.out.println("Division By Zero");
		}
		catch(ArrayIndexOutOfBoundsException e)
		{
			System.out.println("Array Index Error");
		}

		int y=a[1]/a[0];
		System.out.println("Y = "+y);
	}
}
```

The finally block

The finally block follows a try block or a catch block.

A finally block of code always executes, irrespective of occurrence of an Exception.

Using a finally block allows you to run any cleanup-type statements that you want to execute, no matter what happens in the protected code.

A finally block appears at the end of the catch blocks and has the following syntax:

```
try
{
 //Protected code
 }
catch(ExceptionType1 e1)
{
 //Catch block
}
finally
 {
 //The finally block always executes.
}
```

```
public class ex50
{
    public static void main(String Args[])
    {
     int[] array = new int[3];
     try
     {
          for(int i=0;i<4;++i)
          {
               array[i] = i;
          }
          for(int i=0;i<4;++i)
          {
               System.out.println(array[i]);
          }
```

```
catch(ArrayIndexOutOfBoundsException e)
        {
                //printed just to inform that we have entered the catch block
                System.out.println("Exception is generated..");
        }
        finally
        {
                System.out.println("We are in finally Inner");
        }


        System.out.println("We are in finally outer");


    }
}
```

Throwing our own Exception

Sometimes we need to throw our own exception we can do this by using the keyword “throw”.

Syntax :

throw new <throwable subclass>

Example :

throw new ArithmeticException();

We can also create our own class for Exception which must extends the Exception class.

Java throw keyword

The Java throw keyword is used to explicitly throw an exception.

We can throw either checked or uncheked exception in java by throw keyword. The throw keyword is mainly used to throw custom exception. We will see custom exceptions later.

The syntax of java throw keyword is given below.

throw exception;

Let's see the example of throw IOException.

throw new IOException("sorry device error);

```
public class TestThrow1{
   static void validate(int age){
     if(age<18)
      throw new ArithmeticException("not valid");
     else
      System.out.println("welcome to vote");
   }
   public static void main(String args[]){
      validate(13);
      System.out.println("rest of the code...");
  }
}
```

Output:

Exception in thread main

java.lang.ArithmeticException: not valid

Java throws keyword

The **Java throws keyword** is used to declare an exception.

It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Exception Handling is mainly used to handle the checked exceptions.

If there occurs any unchecked exception such as NullPointerException, it is programmers fault that he is not performing check up before the code being used.

Syntax of java throws

```
return_type method_name() throws exception_class_name
{
    //method code
}
```

```
import java.io.*;
class ThrowsEx
{
    public static void main(String args[]) throws IOException,ArithmeticException
    {
     int a=0,b=0,sum=0;
     DataInputStream dis=new DataInputStream(System.in);
     System.out.print("Enter Value 1 : ");
     a=Integer.parseInt(dis.readLine());
     System.out.print("Enter Value 2 : ");
     b=Integer.parseInt(dis.readLine());
     sum=a+b;
     System.out.println("Sum is : "+sum);
    }
}
```

Which exception should be declared ?

Ans) checked exception only, because:

unchecked Exception is under your control so you can make your code correct.

Error is beyond your control e.g. you are unable to do anything if it occurs.

Custom Exception

The term exception lets the programmer to know that there is something exceptional has occurred in the program.

Apart from Java exception class library, a custom exception can be created by the developer.

This customization gives the power to deal with application centric exceptions.

For instance, in a banking application a customer tries to withdraw amount which results the minimum balance.

In this scenario, the developer needs to implement a customized exception.

Difference between throw and throws in Java

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

What is chained exceptions in java?

Once an application finds an exception, responds an exception by throwing another exception.

It causes another exception. Knowing an exception that causes another exception is very useful.

Exception Chaining

Whenever in a program the first exception causes an another exception, that is termed as Chained Exception.

Java provides new functionality for chaining exceptions. Exception chaining (also known as "nesting exception") is a technique for handling the exception, which occur one after another i.e. most of the time is given by an application to response to an exception by throwing another exception.

Typically the second exception is caused by the first exception.

Therefore chained exceptions help the programmer to know when one exception causes another.

Exception Chaining

The syntax for using a chained exception is as follows in which a new TestException exception is created with the attached cause when an IOException is caught.

Thus the chain exception is thrown to next level of exception handler.

```
try
{  }
catch (IOException e)
{
   throw new TestException("Other IOException", e);
}
```

Exception Chaining

```
import java.io.*;
class MyException extends Exception
{
    MyException(String msg)
    {
     super(msg);
    }
}
```

```
public class ex49
{
    public static void main(String args[]) throws MyException, IOException, ArithmeticException
    {
        try
        {
            int rs=10/0;
        }
            catch(Exception e)
             {
            System.out.println(e.getMessage());
```

Exception Chaining

```
        try
        {
            throw new MyException("Chained ArithmeticException");

        }
        catch(Exception e1)
        {
            System.out.println(e1.getMessage());
        }

        }
    }
}
```

Thank You